

"""

=====

GENERADOR DE REPORTES AFIP / ARCA – MoliPay / Money Life S.R.L.

=====

Régimen Informativo RG (AFIP) 4614/2019 y modif. Genera los archivos TXT que deben presentarse por el servicio "Presentación de DDJJ y Pago" o vía Webservice PRESENTACIÓN DE DDJJ.

F.8125 – Título I – Agrupadores/Agregadores de medios de pago

F.8126 – Título II – Billeteras virtuales / cuentas virtuales (V300)

#### FORMATO CRÍTICO

-----

- Archivo TXT de texto plano
- Encoding ISO-8859-1 (NO utf-8)
- Sin separadores. Cada campo va en posición fija
- Alfanuméricos: alineados a IZQUIERDA, rellenos con ESPACIOS a la derecha
- Numéricos: alineados a DERECHA, rellenos con CEROS a la izquierda
- Sin decimales, sin separadores de miles
- Sin líneas en blanco
- Salto de línea: LF (0x10) o CRLF (0x10 0x13); no incluir otros controles

Nombre del archivo:

F{form}.{CUIT}.{YYYYMM}00.{secuencia4}.txt

Ejemplos:

F8125.30999999995.20260800.0000.txt

F8126.30999999995.20260800.0000.txt

Referencias oficiales:

Manual F.8125 – <https://www.afip.gob.ar/economia-digital/ayuda/>

documentos/Manual-usuario-F8125.pdf

Manual F.8126 V300 – <https://www.afip.gob.ar/economia-digital/ayuda/>

documentos/Manual-usuario-F8126V300.pdf

=====

"""

```
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Iterable
from datetime import datetime
```

```
# =====
# UTILIDADES DE FORMATO – TODAS las líneas se arman con estas funciones
```

```

# =====

def num(valor, largo: int) -> str:
    """Numérico: derecha, ceros a izquierda, sin decimales ni separadores."""
    if valor is None or valor == "":
        valor = 0
    s = str(int(valor))
    if len(s) > largo:
        raise ValueError(f"Numérico excede largo {largo}: {s}")
    return s.rjust(largo, "0")

def alfa(valor, largo: int) -> str:
    """Alfanumérico: izquierda, espacios a derecha."""
    if valor is None:
        valor = ""
    s = str(valor)
    # Reemplazar caracteres de control C0/C1 que AFIP rechaza
    s = "".join(c if (32 <= ord(c) < 127) or (160 <= ord(c) <= 255) else " "
                for c in s)
    if len(s) > largo:
        s = s[:largo]
    return s.ljust(largo, " ")

def signo(monto: int) -> str:
    """0=positivo, 1=negativo (AFIP guarda el signo separado del valor)."""
    return "1" if monto < 0 else "0"

def abs_num(monto, largo: int) -> str:
    """Valor absoluto formateado como numérico."""
    return num(abs(int(monto or 0)), largo)

# =====
# =====
# F.8125 – Agregadores / Agrupadores de medios de pago
# =====
# =====
# Estructura de 3 tipos de registro anidados:
#
# 01 Cabecera (largo 261)
# 02 Vendedor o Prestador (largo 42) ← uno por comercio
# 03 Detalle de operaciones (largo 41) ← uno o más por vendedor
#
# El árbol: 01 → 02 → 03 [03 03...] → 02 → 03 [03 03...] → 02 ...

```

```

# =====

@dataclass
class OperacionF8125:
    """Un detalle de operación (Registro Tipo 03) dentro de un vendedor."""
    metodologia: str          # "01"=CBU "02"=CVU "03"=EFECTIVO "04"=CHEQUE "05"=OT
    monto: int                # positivo o negativo; el signo se separa
    tipo_cuenta: str = "00" # tabla Cuentas; obligatorio solo si metodologia="01"
    nro_identificacion: str = "0" # CBU/CVU si corresponde; sino "0"

@dataclass
class VendedorF8125:
    """Un vendedor/prestador (Registro Tipo 02) con sus operaciones."""
    tipo_doc: str             # "80"=CUIT "86"=CUIL "87"=CDI
    cuit: str                 # 11 dígitos sin guiones
    codigo_rubro: str         # "01"... "08" según tabla Rubros
    monto_total_mes: int      # sumatoria de operaciones del mes
    comision_cobrada: int
    operaciones: list[OperacionF8125] = field(default_factory=list)

def generar_f8125(
    cuit_informante: str,
    periodo: str,             # "YYYYMM"
    denominacion: str,
    vendedores: list[VendedorF8125],
    secuencia: int = 0,       # 0=original, >0=rectificativa
    numero_verificador: int = 0,
    ruta_salida: str | None = None,
) -> str:
    """
    Devuelve el contenido del archivo como string. Si ruta_salida está dado,
    lo escribe en ISO-8859-1.

    Si no hay vendedores → presentación SIN MOVIMIENTOS (solo cabecera).
    """
    lineas: list[str] = []

    # Contar registros de detalle (02 + 03). Si es 0 → SIN MOVIMIENTOS = 1
    total_detalle = sum(1 + len(v.operaciones) for v in vendedores)
    cantidad_detalle = total_detalle if total_detalle > 0 else 1

    # ----- REGISTRO 01 - CABECERA (largo 261) -----
    ahora = datetime.now().strftime("%H%M%S")
    cabecera = (
        "01"                                # 1-2   Tipo registro
        + num(cuit_informante, 11)          # 3-13  CUIT informante

```

```

+ num(periodo, 6) # 14-19 Período AAAAMM
+ num(sequencia, 2) # 20-21 Secuencia
+ alfa(denominacion, 200) # 22-221 Denominación
+ num(ahora, 6) # 222-227 HHMMSS
+ "0103" # 228-231 Cód. impuesto (fijo)
+ "830" # 232-234 Cód. concepto (fijo)
+ num(numero_verificador, 6) # 235-240 N° verificador
+ "8125" # 241-244 N° formulario
+ "00100" # 245-249 Versión aplicativo
+ "00" # 250-251 Establecimiento
+ num(cantidad_detalle, 10) # 252-261 Cant. registros detalle
)

assert len(cabecera) == 261, f"Cabecera F.8125 mal formada: {len(cabecera)}"
lineas.append(cabecera)

# ----- REGISTROS 02 y 03 -----
for v in vendedores:
    # Registro 02 - Vendedor (largo 42)
    r02 = (
        "02" # 1-2
        + num(v.tipo_doc, 2) # 3-4
        + num(v.cuit, 11) # 5-15
        + num(v.codigo_rubro, 2) # 16-17
        + signo(v.monto_total_mes) # 18
        + abs_num(v.monto_total_mes, 12) # 19-30
        + abs_num(v.comision_cobrada, 12) # 31-42
    )
    assert len(r02) == 42, f"R02 F.8125 mal formado: {len(r02)}"
    lineas.append(r02)

# Validación de negocio: la suma de detalles debe igualar monto_total
suma_ops = sum(op.monto for op in v.operaciones)
if suma_ops != v.monto_total_mes:
    raise ValueError(
        f"Vendedor CUIT {v.cuit}: suma de detalles ({suma_ops}) != "
        f"monto_total_mes ({v.monto_total_mes})"
    )

for op in v.operaciones:
    r03 = (
        "03" # 1-2
        + num(op.metodologia, 2) # 3-4
        + num(op.tipo_cuenta, 2) # 5-6
        + num(op.nro_identificacion, 22) # 7-28
        + signo(op.monto) # 29
        + abs_num(op.monto, 12) # 30-41
    )

```

```

        assert len(r03) == 41, f"R03 F.8125 mal formado: {len(r03)}"
        lineas.append(r03)

    contenido = "\n".join(lineas) + "\n"

    if ruta_salida:
        with open(ruta_salida, "w", encoding="iso-8859-1", newline="") as f:
            f.write(contenido)

    return contenido

# =====
# =====
# F.8126 V300 – Billeteras / cuentas virtuales
# =====
# =====
# Estructura JERÁRQUICA de 6 tipos anidados. Orden estricto:
#
# 01  Cabecera                (largo 259)
# 02  Titular de cuenta       (largo 203)
# 03  Detalle cuenta asociada (largo 540)
# 04  Otros integrantes       (largo 102) ← solo si más de 1 integrant
# 05  Movimientos mensuales   (largo 34)  ← uno por (tipo op, moneda)
# 06  Detalle transferencias   (largo 36)  ← solo si supera umbral
#
# La concatenación es POR CUENTA, no por bloques:
# R3 cta1 + R4 cta1 + R5 cta1 + R6 cta1 + R3 cta2 + R4 cta2 + R5 cta2 ...
# =====

@dataclass
class DetalleTransferenciaF8126:
    """Registro Tipo 06 – Detalle de transferencia sobre umbral."""
    cbu_cvu: str          # 22 dígitos
    monto_pesos: int

@dataclass
class MovimientoF8126:
    """Registro Tipo 05 – Movimiento mensual (una fila por tipo+moneda)."""
    tipo_operacion: str    # "01"=INGRESO "02"=EGRESO
    detalle_operacion: str # "01"=Efectivo ... "09"=Otros (ver manual)
    moneda: str           # "01"=PESOS "02"=USD ... "42"=Otro cripto
    monto_moneda_original: int
    monto_pesos: int
    detalles_transferencia: list[DetalleTransferenciaF8126] = field(default_factory=list)

@dataclass

```

```

class IntegranteF8126:
    """Registro Tipo 04 - Cotitular/apoderado/etc."""
    tipo_persona: int          # 1=Humana, 2=Jurídica
    caracter: str               # "01"=Titular "02"=Cotitular ... "06"=Otros
    nacionalidad: str           # "N"=Argentina "E"=Extranjera
    pais: str = "200"           # tabla países (200=Arg)
    tipo_doc: str = "80"        # "80"=CUIT ... "99"=NIF
    cuit: str = ""              # si tipo_doc en (80,86,87)
    otro_doc: str = ""          # si tipo_doc en (88,94,99)
    apellido_nombre: str = ""

@dataclass
class CuentaAsociadaF8126:
    """Registro Tipo 03 - Una cuenta asociada al titular."""
    tipo_cuenta: str            # "01"=CVU/CBU "02"=Otra
    cvu_cbu: str = "0"          # obligatorio si tipo_cuenta="01"
    id_otro_tipo: str = ""      # obligatorio si tipo_cuenta="02"
    cantidad_integrantes: int = 1
    denominacion_emisora: str = "" # solo si otorgante ≠ informante
    tipo_doc_emisora: str = ""
    nro_doc_emisora: str = ""
    cta_por_cuenta_terceros: int = 0 # 0=NO 1=SI (para PSPCP × PSAV)
    denominacion_tercero: str = "" # obligatorio si cta_por_cuenta_terceros=1
    saldo_pesos: int = 0
    saldo_moneda_extranjera_en_pesos: int = 0
    saldo_cripto_en_pesos: int = 0
    otros_integrantes: list[IntegranteF8126] = field(default_factory=list)
    movimientos: list[MovimientoF8126] = field(default_factory=list)

@dataclass
class TitularF8126:
    """Registro Tipo 02 - Titular de cuenta en la plataforma."""
    tipo_persona: int          # 1=Humana, 2=Jurídica
    nacionalidad: str           # "N" o "E"
    pais: str                   # "200"=Arg
    tipo_doc: str               # "80"=CUIT etc.
    cuit: str = ""
    otro_doc: str = ""
    apellido_nombre: str = ""
    id_cliente: str = ""        # legajo interno; si no existe → CUIT/CUIL/CDI
    fecha_alta: str = ""        # YYYYMMDD
    tipo_operacion: str = "02" # "01"=Cierre "02"=Con mov "03"=Sin mov
    saldo_pesos: int = 0
    saldo_moneda_extranjera_en_pesos: int = 0
    saldo_cripto_en_pesos: int = 0
    cuentas_asociadas: list[CuentaAsociadaF8126] = field(default_factory=list)

```

```

def generar_f8126(
    cuit_informante: str,
    periodo: str,
    denominacion: str,
    titulares: list[TitularF8126],
    secuencia: int = 0,
    numero_verificador: int = 0,
    ruta_salida: str | None = None,
) -> str:
    """
    Genera el .txt F.8126 V300. Sin titulares → SIN MOVIMIENTOS.
    """
    lineas: list[str] = []

    # ---- Contar todos los registros de detalle (02+03+04+05+06) -----
    total = 0
    for t in titulares:
        total += 1 # R02
        for c in t.cuentas_asociadas:
            total += 1 # R03
            # R04 solo si cantidad_integrantes > 1 (se informan los "otros")
            if c.cantidad_integrantes > 1:
                total += len(c.otros_integrantes)
            for m in c.movimientos:
                total += 1 # R05
                total += len(m.detalles_transferencia) # R06
    cantidad_detalle = total if total > 0 else 1

    # ---- REGISTRO 01 CABECERA (largo 259) -----
    ahora = datetime.now().strftime("%H%M%S")
    cabecera = (
        "01"
        + num(cuit_informante, 11)
        + num(periodo, 6)
        + num(secuencia, 2)
        + alfa(denominacion, 200)
        + num(ahora, 6)
        + "0103" # Cód. impuesto
        + "812" # Cód. concepto (F.8126 = 812)
        + num(numero_verificador, 6)
        + "8126"
        + "00300" # Versión 300
        + num(cantidad_detalle, 10)
    )
    assert len(cabecera) == 259, f"Cabecera F.8126 mal formada: {len(cabecera)}"
    lineas.append(cabecera)

```

```

# ---- Ciclo TITULAR → CUENTA → INTEGRANTES → MOVIMIENTOS → TRANSFERS ----
for t in titulares:
    r02 = (
        "02" # 1-2
        + num(t.tipo_persona, 1) # 3
        + alfa(t.nacionalidad, 1) # 4
        + alfa(t.pais, 3) # 5-7
        + num(t.tipo_doc, 2) # 8-9
        + (num(t.cuit, 11) if t.cuit else " " * 11) # 10-20
        + alfa(t.otro_doc, 20) # 21-40
        + alfa(t.apellido_nombre, 60) # 41-100
        + alfa(t.id_cliente or t.cuit, 50) # 101-150
        + num(t.fecha_alta, 8) # 151-158
        + num(t.tipo_operacion, 2) # 159-160
        + signo(t.saldo_pesos) # 161
        + abs_num(t.saldo_pesos, 12) # 162-173
        + signo(t.saldo_moneda_extranjera_en_pesos) # 174
        + abs_num(t.saldo_moneda_extranjera_en_pesos, 12) # 175-186
        + signo(t.saldo_cripto_en_pesos) # 187
        + abs_num(t.saldo_cripto_en_pesos, 12) # 188-199
        + num(len(t.cuentas_asociadas), 4) # 200-203
    )
    assert len(r02) == 203, f"R02 F.8126 mal formado: {len(r02)}"
    lineas.append(r02)

for c in t.cuentas_asociadas:
    r03 = (
        "03" # 1-2
        + num(c.tipo_cuenta, 2) # 3-4
        + alfa(c.cvu_cbu, 22) # 5-26
        + alfa(c.id_otro_tipo, 50) # 27-76
        + num(c.cantidad_integrantes, 2) # 77-78
        + alfa(c.denominacion_emisora, 200) # 79-278
        + alfa(c.tipo_doc_emisora, 2) # 279-280
        + alfa(c.nro_doc_emisora, 20) # 281-300
        + num(c.cta_por_cuenta_terceros, 1) # 301
        + alfa(c.denominacion_tercero, 200) # 302-501
        + signo(c.saldo_pesos) # 502
        + abs_num(c.saldo_pesos, 12) # 503-514
        + signo(c.saldo_moneda_extranjera_en_pesos) # 515
        + abs_num(c.saldo_moneda_extranjera_en_pesos, 12) # 516-527
        + signo(c.saldo_cripto_en_pesos) # 528
        + abs_num(c.saldo_cripto_en_pesos, 12) # 529-540
    )
    assert len(r03) == 540, f"R03 F.8126 mal formado: {len(r03)}"
    lineas.append(r03)

```



```

# R04 – otros integrantes (solo si hay más de 1)
if c.cantidad_integrantes > 1:
    for i in c.otros_integrantes:
        r04 = (
            "04"                                # 1-2
            + num(i.tipo_persona, 1)             # 3
            + num(i.caracter, 2)                 # 4-5
            + alfa(i.nacionalidad, 1)            # 6
            + alfa(i.pais, 3)                    # 7-9
            + num(i.tipo_doc, 2)                 # 10-11
            + (num(i.cuit, 11) if i.cuit else " " * 11) # 12-22
            + alfa(i.otro_doc, 20)               # 23-42
            + alfa(i.apellido_nombre, 60)       # 43-102
        )
        assert len(r04) == 102, f"R04 F.8126 mal formado: {len(r04)}"
        lineas.append(r04)

# R05 – movimientos mensuales
for m in c.movimientos:
    r05 = (
        "05"                                # 1-2
        + num(m.tipo_operacion, 2)           # 3-4
        + num(m.detalle_operacion, 2)        # 5-6
        + num(m.moneda, 2)                   # 7-8
        + num(m.monto_moneda_original, 13)   # 9-21
        + num(m.monto_pesos, 13)            # 22-34
    )
    assert len(r05) == 34, f"R05 F.8126 mal formado: {len(r05)}"
    lineas.append(r05)

# R06 – detalle transferencias (solo sobre umbral)
for d in m.detalles_transferencia:
    r06 = (
        "06"                                # 1-2
        + alfa(d.cbu_cvu, 22)               # 3-24
        + num(d.monto_pesos, 12)            # 25-36
    )
    assert len(r06) == 36, f"R06 F.8126 mal formado: {len(r06)}"
    lineas.append(r06)

contenido = "\n".join(lineas) + "\n"

if ruta_salida:
    with open(ruta_salida, "w", encoding="iso-8859-1", newline="") as f:
        f.write(contenido)

```

```

    return contenido

# =====
# NOMBRE DE ARCHIVO CANÓNICO
# =====

def nombre_archivo(formulario: str, cuit: str, periodo: str, secuencia: int = 0)
    """F8126.30999999995.20260800.0000.txt"""
    return f"F{formulario}.{cuit}.{periodo}00.{secuencia:04d}.txt"

# =====
# MAPEO DESDE LA BASE DE MoliPay (para el programador)
# =====
#
# --- F.8125 (si corresponde presentarlo) ---
# 1 vendedor por CUIT de comercio con actividad en el mes:
#     SELECT comercio_cuit, rubro,
#           SUM(monto) AS monto_mes,
#           SUM(comision) AS comision
#     FROM operaciones_cobro
#     WHERE fecha BETWEEN :inicio_mes AND :fin_mes
#     GROUP BY comercio_cuit, rubro
#
# Por vendedor, un detalle 03 por cada CVU/CBU de acreditación:
#     SELECT metodologia, tipo_cuenta, cbu_cvu, SUM(monto)
#     FROM acreditaciones
#     WHERE comercio_cuit = :cuit AND mes = :periodo
#     GROUP BY metodologia, tipo_cuenta, cbu_cvu
#
# --- F.8126 V300 ---
# Filtrar SOLO usuarios cuyo (ingresos+egresos totales del mes) O (saldo final
# al último día hábil del mes) supere el umbral vigente del art. 3° RG 4614.
# Sumar saldos en ARS + FX + cripto para el test del umbral.
#
# Por usuario (R02): datos identificatorios + saldo final por moneda.
# Por cada cuenta interna/asociada (R03): CVU propio, cuentas comitentes,
#     CBU's bancarios vinculados, saldos.
#     Si actúan por cuenta y orden de un PSAV, campo 8=1 y denominar en 9.
# Por cuenta con >1 integrante (R04): cotitulares, apoderados, etc.
# Por cuenta con movimientos (R05): agrupar por (tipo_op × detalle × moneda).
# Por movimiento tipo Transferencia (02 o 03) en moneda fiat (01-13) sobre
#     umbral 5% del art. 3° RG 4614 → un R06 por CBU/CVU contraparte.
#
# =====
# EJEMPLO DE USO

```

```

# =====

if __name__ == "__main__":

    # ----- F.8125 (ejemplo con 1 vendedor y 2 operaciones) -----
    vendedores = [
        VendedorF8125(
            tipo_doc="80",
            cuit="30712345678",
            codigo_rubro="04",      # Restaurantes
            monto_total_mes=1_500_000,
            comision_cobrada=45_000,
            operaciones=[
                OperacionF8125(
                    metodologia="01",    # CBU
                    tipo_cuenta="10",    # Cta cte con interés
                    nro_identificacion="2850590940090418135201",
                    monto=1_000_000,
                ),
                OperacionF8125(
                    metodologia="02",    # CVU
                    tipo_cuenta="00",
                    nro_identificacion="00000031000100000000123",
                    monto=500_000,
                ),
            ],
        ),
    ]

    generar_f8125(
        cuit_informante="309999999995",
        periodo="202608",
        denominacion="MONEY LIFE S.R.L.",
        vendedores=vendedores,
        ruta_salida=nombre_archivo("8125", "309999999995", "202608"),
    )

    # ----- F.8126 (ejemplo con 1 titular, 1 cuenta CVU, movimientos) ----
    titulares = [
        TitularF8126(
            tipo_persona=1,
            nacionalidad="N",
            pais="200",
            tipo_doc="80",
            cuit="20345678901",
            apellido_nombre="PEREZ, JUAN CARLOS",
            id_cliente="ML-0001234",
            fecha_alta="20240315",

```

```

        tipo_operacion="02",
        saldo_pesos=234_567,
        cuentas_asociadas=[
            CuentaAsociadaF8126(
                tipo_cuenta="01",
                cvu_cbu="00000031000100000000123",
                cantidad_integrantes=1,
                saldo_pesos=234_567,
                movimientos=[
                    MovimientoF8126(
                        tipo_operacion="01",    # Ingreso
                        detalle_operacion="02", # Transferencia de terceros
                        moneda="01",           # Pesos
                        monto_moneda_original=500_000,
                        monto_pesos=500_000,
                    ),
                    MovimientoF8126(
                        tipo_operacion="02",    # Egreso
                        detalle_operacion="08", # Pago con transferencia
                        moneda="01",
                        monto_moneda_original=265_433,
                        monto_pesos=265_433,
                    ),
                ],
            ),
        ],
    ],
)

generar_f8126(
    cuit_informante="309999999995",
    periodo="202608",
    denominacion="MONEY LIFE S.R.L.",
    titulares=titulares,
    ruta_salida=nombre_archivo("8126", "309999999995", "202608"),
)

print("Archivos generados:")
print(" ", nombre_archivo("8125", "309999999995", "202608"))
print(" ", nombre_archivo("8126", "309999999995", "202608"))

```